THE DESIGN AND IMPLEMENTATION OF A GYM MANAGEMENT WEBSITE (FITZONE)

By

Zhwan Omar

Bafrin Ali

Pshtiwan Rasul

Dyako Ali

Zaman qurbani

Chopy Hama

Under Supervision of:

Savan

April 2026

Kurdistan Regional Government-Iraq

Ministry of Education

General Directorate of Institutes and Training

Raparin Institute

THE DESIGN AND IMPLEMENTATION OF A GYM MANAGEMENT WEBSITE (FITZONE)

Research Project Submitted in Partial Fulfillment of the Requirements

for the Degree of Diploma in Computer Science

By

Zhwan Omar **Bafrin Ali**

Pshtiwan Rasul **Dyako Ali**

Zaman qurbani **Chopy Hama**

Under Supervision of:

Savan

April 2026

DECLARATION

This research project entitled "The Design and Implementation of a Gym Management Website (FitZone)" has not previously been accepted in substance for any degree and is not being concurrently submitted in candidature for any degree.

STATEMENT 1

This research project entitled "The Design and Implementation of a Gym Management Website (FitZone)" is being submitted in partial fulfillment of the requirements for the Degree of Diploma in Computer Science.

STATEMENT 2

This research project entitled "The Design and Implementation of a Gym Management Website (FitZone)" is the result of our own investigation, except where otherwise stated. Other sources are acknowledged by giving explicit references.

STATEMENT 3

We hereby give consent for our research project entitled "The Design and Implementation of a Gym Management Website (FitZone)", if accepted, to be available for photo-copying and for interlibrary loan, and for the title and summary to be made available to outside organizations.

Name: Bafrin Ali ..... Sign: ..... Date: .....

Name: Pshtiwan Rasul ..... Sign: ..... Date: .....

Name: Zhwan Omar ..... Sign: ..... Date: .....

Name: Dyako ali ..... Sign: ..... Date: .....

Name: Chopy hama ..... Sign: ..... Date: .....

Name: Zamand Qurbani ..... Sign: ..... Date: .....

SUPERVISOR'S CERTIFICATION

I certify that this research project entitled "The Design and Implementation of a Gym Management Website (FitZone)" was prepared under my supervision at Raparin Institute, in partial fulfillment of the requirement for the Degree of Diploma in Computer Science.

Signature: .....

Name: .....

Date: .....

REPORT OF THE DIRECTOR OF THE INSTITUTE

According to the recommendation submitted by the supervisor of this project, I nominate this research project to be forwarded for discussion.

Signature: .....

Name: .....

Date: .....

Examination Committee Certification

We certify that the information provided in this research project is complete and correct, the research has been approved on behalf of the institute research review committee.

دانپیدانانی لیژنەی تاقیکردنەوە

پشتڕاست دەکەینەوە کە زانیاریەکانی ناو ئەم پرۆژەی توێژینەوەیە تەواو و دروستە، و توێژینەوەکە لەلایەن لیژنەی پێداچوونەوەی توێژینەوەی پەیمانگاوە پەسەند کراوە.

Signature: ..... Name: ..... Date: ..... (Member)	Signature: ..... Name: ..... Date: ..... (Member)
Signature: ..... Name: ..... Date: ..... (Supervisor)	Signature: ..... Name: ..... Date: ..... (Member)
Approved by the Institute Committee of Graduate Studies	

Approved by the Institute Committee of Graduate Studies
Signature: ..... Name: ..... Date: ..... Signature: ..... Name: ..... Date: .....

ABSTRACT

When we first started thinking about what to build for our graduation project, we kept going back and forth between different ideas. Eventually we landed on a gym website because, honestly, most gyms around here in the Kurdistan Region don't really have a proper online system. Some of them just use Instagram pages or WhatsApp groups to communicate with members, which felt like something we could actually improve on.

So we built FitZone - it's basically a website that lets a gym manage pretty much everything online. Members can sign up, browse through exercises, check out trainers, look at different membership plans, and so on. We also built a dashboard where logged-in users can save their own workout lists and write notes about their daily training, like how they felt and what their weight was that day.

On the backend, there's a full admin panel where the gym owner or manager can control everything. They can add or edit exercises, manage trainers, update membership plans, handle contact messages from visitors, and change settings like the site colors and text. One thing we're pretty proud of is that the whole site works in both Kurdish and English, and when you switch to Kurdish the layout flips to right-to-left which was honestly harder to get right than we expected.

We used PHP for the server side, MySQL for the database, and Bootstrap 5 for making the design look decent and work on phones too. The database has 17 tables in total and we tried our best to keep things secure with stuff like password hashing, CSRF tokens, and prepared statements so nobody can do SQL injection. It's not perfect but it works well enough for what it needs to do.

پوختە

کۆتاییدا لە بەڵام هەبوو بیرۆکەمان چەندین دەکردینەوە، دەرچوونمان پڕۆژەی لە بیر بووین خەریک کاتێک هۆڵە زۆربەی کە بوو ئەوە هۆکارەکەشی .بکەین دروست وەرزشی هۆڵی وێبسایتێکی دا بڕیارمان دەکەن، کار واتسئاپەوە یان ئینستاگرام ڕێگەی لە تەنها و نییە ئۆنلاینیان سیستەمی ناوچەکەمان وەرزشیەکانی .بکرێت باشتر ئەتوانرێت ئێمە بۆچوونی بە کە

بە شتێک هەموو دەدات وەرزشی هۆڵی بە ڕێگا کە وێبسایتێکە - ـە FitZone ناوی دروستمانکرد ئەوەی و بناسن، ڕاهێنەرەکان ببینن، ڕاهێنانەکان بکەن، تۆمار خۆیان دەتوانن ئەندامەکان .ببات بەڕێوە ئۆنلاین ژوورەوە چوویەتە بەکارهێنەری کە دروستکرد داشبۆردێکمان هەروەها .بکەن سەیر ئەندامێتی پلانەکانی و جەستەیی بارودۆخی وەک بنووسێت ڕۆژانە تێبینی و بکات دروست خۆی ڕاهێنانی لیستی دەتوانێت .کێشەکەی

بەڕێوەبەرەکەی یان هۆڵەکە خاوەنی کە دروستکرد تەواومان بەڕێوەبردنی پانێلێکی ئەدمینەوە، لایەنی لە نوێ پلانەکان ببەن، بەڕێوە ڕاهێنەرەکان بکەن، زیاد ڕاهێنان دەتوانن .بکات کۆنترۆل شتێک هەموو دەتوانێت کە ئەوەیە دەکات خۆشحاڵمان زۆر کە شتێک .بگۆڕن سایتەکە ڕێکخستنەکانی و ببینن، نامەکان بکەنەوە، ڕووکارەکە دەگۆڕیت کوردی بۆ کاتێک و دەکات کار ئینگلیزی و کوردی زمانی هەردوو بە وێبسایتەکە .چەپ بۆ ڕاست بۆ دەگەڕێتەوە

.دیزاین بۆ Bootstrap 5 و داتابەیس، بۆ MySQL سێرڤەر، بۆ بوو PHP بەکارهێنراو تەکنەلۆژیاکانی و CSRF تۆکنی و نهێنی وشەی هاشکردنی بە بپارێزین ئەمنیەت هەوڵمانداوە و تێدایە خشتەی ١٧ داتابەیسەکە .SQL هێرشی لە ڕێگری بۆ ئامادەکراو ڕستەی

ACKNOWLEDGEMENTS

Before anything else, we want to say a big thank you to Raparin Institute for everything. These two years have taught us so much, not just about computers and coding but about working in a team and solving problems on our own. We're really grateful for this chance.

A huge thanks goes to our supervisor who was patient with us when we got stuck (which happened more often than we'd like to admit). Every time we ran into a wall, they pointed us in the right direction without just giving us the answer, and that honestly helped us learn way more than if they had just told us what to do.

We also want to thank the other teachers in the Computer Science department. Each one of them taught us something that ended up being useful in this project, whether it was database design, programming basics, or even just how to think about a problem logically.

To our families - thank you for putting up with us during the late nights and the weekends we spent glued to our laptops. Your support and encouragement kept us going, especially during the times when nothing seemed to be working and we felt like starting over.

We should also mention the online developer community. Stack Overflow, YouTube tutorials, and various PHP and Bootstrap documentation pages saved us countless times. It's pretty amazing how much free knowledge is out there for anyone willing to look for it.

And finally, thank God for giving us the health, patience, and ability to finish this project. It wasn't always easy, but we made it through.

LIST OF ACRONYMS

Acronym	Description
PHP	Hypertext Preprocessor
HTML	Hypertext Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
SQL	Structured Query Language
MySQL	My Structured Query Language
PDO	PHP Data Objects
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
XSS	Cross-Site Scripting
BCRYPT	Blowfish Crypt
MVC	Model-View-Controller
RTL	Right-To-Left
LTR	Left-To-Right
CDN	Content Delivery Network
AJAX	Asynchronous JavaScript and XML
API	Application Programming Interface
URL	Uniform Resource Locator
JSON	JavaScript Object Notation
OOP	Object-Oriented Programming
XAMPP	Cross-platform Apache MySQL PHP Perl
UTF-8	Unicode Transformation Format - 8 bit
SEO	Search Engine Optimization
UI	User Interface
UX	User Experience

LIST OF FIGURES

Index	Figure Title
Figure 3.1	System Architecture Diagram
Figure 3.2	Database Entity-Relationship Diagram
Figure 3.3	Website Flowchart
Figure 3.4	Home Page Design
Figure 3.5	User Registration Page
Figure 3.6	User Login Page
Figure 3.7	Exercises Page with Filters
Figure 3.8	Trainers Page
Figure 3.9	Membership Plans Page
Figure 3.10	User Dashboard
Figure 3.11	Admin Dashboard
Figure 3.12	Admin Games Management
Figure 3.13	Admin Settings Page
Figure 3.14	Mobile Responsive View
Figure 3.15	Contact Page
Figure 3.16	Tips/Blog Page
Figure 3.17	Admin Trainers Management
Figure 3.18	Admin Plans Management
Figure 3.19	Admin Messages Page
Figure 3.20	User Workout Lists
Figure 3.21	User Daily Notes
Figure 3.22	User Profile Page
Figure 3.23	Admin Color Management
Figure 3.24	Kurdish Language View (RTL)

LIST OF TABLES

Index	Table Title
Table 2.1	Comparison of Web Technologies
Table 2.2	Software Requirements
Table 2.3	Hardware Requirements
Table 3.1	Project Plan and Timeline
Table 3.2	Database Tables Overview
Table 3.3	Users Table Structure
Table 3.4	Games Table Structure
Table 3.5	User Features Summary
Table 3.6	Admin Panel Features Summary

Contents

DECLARATION 5

SUPERVISOR'S CERTIFICATION 7

REPORT OF THE DIRECTOR OF THE INSTITUTE 7

Examination Committee Certification 8

ABSTRACT 10

ACKNOWLEDGEMENTS 12

LIST OF ACRONYMS 13

LIST OF FIGURES 14

LIST OF TABLES 15

CONTENTS ..... **Error! Bookmark not defined.**

CHAPTER ONE 19

INTRODUCTION 19

1.1 Introduction 19

1.2 Aims 20

1.3 Project Overview 21

CHAPTER TWO 23

THEORETICAL BACKGROUND 23

2.1 Literature Review 23

2.2 Methodology 24

2.3 Project Requirements 25

2.3.1 Software Requirements 25
2.3.2 Hardware Requirements 26
2.3.3 Technologies Used 26
CHAPTER THREE 28
DESIGN AND IMPLEMENTATION 28
3.1 Project Plan 28
3.2 Project Design 29
3.2.1 System Architecture 29
3.2.2 System Flowchart 30
3.2.3 Database Design 31
3.3 Project Implementation 34
3.3.1 Database Configuration and Connection 34
3.3.2 Authentication System 35
3.3.3 Public Website Pages 36
3.3.4 User Dashboard 42
3.3.5 Administrative Panel 43
3.3.6 Bilingual Support System 47
3.3.7 Security Implementation 48
3.3.8 Responsive Design 49
3.4 Project Publishing 49
CHAPTER FOUR 51
CONCLUSION AND FUTURE WORK 51
4.1 Conclusion 51

4.2 Future Works 52

REFERENCES 54

CHAPTER ONE

INTRODUCTION

1.1 Introduction

Over the last few years, going to the gym has become a pretty big deal here in Kurdistan. You can see new fitness centers opening up all the time, and more people are getting interested in working out and staying healthy. But here's the thing - while the gyms themselves are getting better, most of them are still running things the old-fashioned way when it comes to their online presence.

What we mean by that is, if you want to know about a gym's membership prices, their trainers, or what kind of exercises they offer, you usually have to either go there in person or message them on social media and wait for a reply. Very few gyms in our area have actual websites, and the ones that do usually just have a basic page with their phone number and address. There's no way for members to log in, track their workouts, or really interact with the gym online at all.

Another thing that bothered us was the language issue. Most website templates and systems out there are in English only. For a gym in Sulaymaniyah or Ranya, having a website that only works in English doesn't make much sense when most of your members speak Kurdish as their first language. We couldn't find any gym management system that properly supports Kurdish with right-to-left layout.

That's basically how FitZone came about. We wanted to create a website that a real gym could actually use - not just a pretty homepage, but a whole system where they can manage their members, show off their exercises and trainers, and give their members tools to plan their own workouts. And we wanted it to work properly in both Kurdish and English because that's what makes sense for our region.

For building this, we went with PHP as our main programming language on the server side. We picked PHP because it's what we've been learning in class and it's still one of the most widely used languages for web development. For the frontend, we used Bootstrap 5 which gave us a modern, responsive design without having to write all the CSS from scratch. And MySQL handles all the database stuff.

The whole system ended up having three main parts. First, there's the public website that anyone can visit - it has a homepage, exercise pages, trainer profiles, membership plans, fitness tips, and a contact form. Second, there's the member dashboard where registered users can create their own workout lists, keep a daily training journal, and manage their profile. And third, there's the admin panel where the gym owner can manage literally everything from exercises to trainers to the colors of the website.

We also put a lot of effort into security because we learned in our classes that web applications can be vulnerable to various attacks. So we implemented things like CSRF tokens on every form, BCRYPT password hashing, prepared SQL statements to prevent injection, and output escaping to prevent XSS. We wanted to make sure that even though this is a student project, it follows real-world security practices.

1.2 Aims

When we sat down to plan this project, we wrote out a list of things we wanted to achieve by the end. Some of these were must-haves, and others were nice-to-haves that we hoped we'd have time for. Here's what we aimed for:

- Build a working gym website that supports both Kurdish and English, with proper RTL layout for Kurdish text. This was the most important one for us because it's what makes our project different from just using a template.
- Create a user registration and login system that's actually secure - with password hashing, session management, and protection against common attacks. We didn't want to just store passwords in plain text like some tutorials show.

- Put together a big exercise database where the admin can add exercises with all the details like difficulty level, which muscles it works, equipment needed, how long it takes, and even YouTube video links for demonstrations.
- Give registered users their own dashboard where they can make custom workout lists, add exercises to those lists with their own sets and reps, keep a daily training journal with mood and weight tracking, and manage their profile.
- Build an admin panel that lets the gym owner manage literally everything - users, exercises, trainers, membership plans, services, tips/articles, certificates, contact messages, and even site-wide settings including colors.
- Make the site look good on phones and tablets too, not just desktop computers. A lot of people browse on their phones these days so this was pretty important.
- Set up a color customization system so the admin can change the look of the website without touching any code. We ended up building a system with over 45 customizable color options.
- Follow good security practices throughout the project including CSRF protection, password hashing with BCRYPT, prepared SQL statements, and proper input validation.

1.3 Project Overview

We've organized this report into four chapters, and here's a quick rundown of what's in each one:

This first chapter, which you're reading now, explains why we chose this project and what we were trying to accomplish. We talked about the problem we saw with gyms not having proper websites, and we listed out all the goals we set for ourselves.

Chapter Two gets into the more technical stuff. We looked at what other people have done in terms of gym management systems, talked about the methodology we

followed (sort of a mix between waterfall and agile), and listed all the software, hardware, and technologies we used.

Chapter Three is the big one - that's where we show the actual work. It covers our project timeline, the system design including the database structure with all 17 tables, and then goes through the implementation of every major component: the public website, user dashboard, admin panel, bilingual system, security features, and responsive design.

Chapter Four wraps everything up. We talk about what we achieved, what challenges we faced along the way, and what we would add or improve if we had more time to keep working on it.

CHAPTER TWO

THEORETICAL BACKGROUND

2.1 Literature Review

Before we started coding anything, we spent some time looking at what already exists out there in terms of gym management systems. We wanted to understand what features people expect and where the gaps are that we might be able to fill.

There are quite a few commercial gym management platforms available. GymMaster is one of the bigger ones - it offers member management, billing, booking, and reporting. Mindbody is another popular option that focuses on scheduling and payments. These are professional products used by thousands of gyms worldwide, but they cost money (usually monthly subscriptions) and they don't support Kurdish or any RTL languages.

We also looked at some open-source alternatives on GitHub. There are a few gym management systems built with PHP/MySQL, but most of them are pretty basic - they handle member registration and maybe billing, but they don't have things like exercise databases with video links, workout list builders, or proper content management for the public website. And again, none of them support RTL languages.

In terms of academic work, we found a study by Ahmed and Hassan (2023) that looked at what fitness centers in the Middle East region actually need from a management system. They found that multilingual support and mobile-friendly design were among the top priorities, which matched what we were planning to build.

On the technology side, PHP is still going strong despite what some people on the internet say about it being "dead." According to W3Techs, as of 2024 about 76.6% of

all websites that use a server-side programming language are using PHP. Big platforms like WordPress, Facebook (originally), and Wikipedia all run on PHP. MySQL is similarly dominant in the database world, especially for web applications.

After looking at all of this, we felt pretty confident that there was a real gap in the market for what we were building. A bilingual, feature-rich gym website that's free and designed specifically for the Kurdistan Region - that's something that doesn't really exist yet.

2.2 Methodology

For how we actually went about building this project, we didn't strictly follow one methodology. In class we learned about Waterfall and Agile, and what we ended up doing was kind of a mix of both. We planned things out in phases (waterfall-ish) but we'd often go back and change things based on testing and feedback (agile-ish).

Here's roughly how it went:

Phase 1 was all about figuring out what we needed to build. We talked to a couple of gym owners we know personally and asked them what would be most useful. We also signed up for free trials of some existing gym management platforms to see what features they offer. This gave us a pretty solid list of requirements.

Phase 2 was the design phase. We drew out the database structure on paper first (old school, we know, but it helped us think it through). We figured out we needed 17 tables to handle everything from users to exercises to settings. We also sketched out the basic page layouts and decided on our color scheme (orange and dark grey as the primary colors).

Phase 3 was the longest phase - actually writing the code. We started with the foundation stuff like database connection, config files, and helper functions. Then we built the public pages, followed by the user dashboard, and finally the admin panel. We worked on it pretty much every day for about 6 weeks.

Phase 4 was testing. We went through every page manually, tried to break things on purpose (like entering weird characters in forms or trying to access admin pages without logging in), and fixed whatever issues we found.

Phase 5 was documentation, which includes writing this very report. We also added code comments to the trickier parts of the codebase so that anyone maintaining the project later would understand what's going on.

2.3 Project Requirements

2.3.1 Software Requirements

Here's everything we used software-wise to build and run FitZone:

Component	Technology / Tool
Operating System	Windows 10/11
Web Server	Apache (via XAMPP 8.2)
Programming Language	PHP 8.2+
Database	MySQL 8.0 / MariaDB 10.4
Frontend Framework	Bootstrap 5.3
CSS	Custom CSS with CSS Variables
JavaScript	Vanilla JavaScript (ES6+)
Development Environment	XAMPP Control Panel
Code Editor	Visual Studio Code
Version Control	Git / GitHub
Browser	Google Chrome / Firefox / Edge
Database Management	phpMyAdmin
Font Libraries	Google Fonts (Inter, Oswald, Noto Sans Arabic)

2.3.2 Hardware Requirements

You don't need a super powerful computer to run this project. Here's the minimum and recommended specs:

Component	Specification
Processor	Intel Core i3 or equivalent (minimum)
RAM	4 GB (minimum), 8 GB (recommended)
Storage	500 MB free disk space for the project
Display	1366x768 resolution (minimum)
Internet	Required for CDN resources and testing

2.3.3 Technologies Used

PHP (Hypertext Preprocessor)

PHP was our main language for the server side. We chose it because it's what we'd been learning in our courses and honestly, it just works well for this kind of project. It handles everything from processing form submissions to querying the database to generating the HTML that gets sent to the browser. We used PHP 8.2 which has some nice improvements like better type support and performance.

MySQL

For the database, MySQL was the obvious choice since it comes bundled with XAMPP and works seamlessly with PHP. Our database ended up having 17 tables, which might sound like a lot but each one serves a specific purpose. We used the PDO extension for database operations because it supports prepared statements natively, which is important for security.

Bootstrap 5

We went with Bootstrap 5 for the frontend styling and it was honestly a lifesaver. Writing all the CSS from scratch would have taken forever, and Bootstrap gives you a solid grid system, nice-looking components, and responsive design essentially for free. We customized it with our own CSS on top to match our brand colors and specific design needs.

JavaScript

We kept the JavaScript simple - no React, no Vue, just plain vanilla JS. It handles stuff like the mobile menu toggle, form validation before submission, the scroll effect on the navigation bar, and a few AJAX calls for things like the search functionality on the exercises page.

XAMPP

XAMPP is what we used to run everything locally on our computers. It bundles Apache (the web server), MySQL (the database), and PHP all together so you don't have to install and configure each one separately. It made development really smooth since you just start XAMPP and everything works.

CHAPTER THREE

DESIGN AND IMPLEMENTATION

3.1 Project Plan

We tried to keep ourselves organized with a rough timeline. It didn't go exactly as planned (does it ever?) but here's more or less how the 14 weeks broke down:

Date	Works
Week 1-2	Talked to gym owners, looked at existing systems, wrote down requirements and planned database structure
Week 3-4	Designed the database schema on paper, then created all 17 tables in MySQL. Set up the project folder structure
Week 5-6	Set up the project foundation - config file, database connection, helper functions, authentication system
Week 7-8	Built all the public pages - homepage, exercises, trainers, plans, tips, contact, certificates
Week 9-10	Developed the admin panel - dashboard with stats, plus management
Week 11-12	Created the user dashboard - workout lists, daily notes, profile management
Week 13	Added Kurdish language support everywhere, implemented RTL layout, tested bilingual functionality
Week 14	Testing, fixing bugs, security review, writing documentation and this report

3.2 Project Design

3.2.1 System Architecture

We organized the project files in a way that keeps things clean and easy to find. It's loosely based on the MVC pattern, though we didn't go full MVC because that would've been overkill for this project. Here's the basic structure:

The root directory has all the public-facing PHP pages that visitors can access directly: index.php for the homepage, login.php and register.php for authentication, games.php for the exercise list, trainers.php for trainer profiles, plans.php for membership options, tips.php for articles, and contact.php for the contact form.

The /includes folder is where the core logic lives. config.php sets up the database connection and site constants. functions.php has all our helper functions - there are over 30 of them covering everything from input sanitization to pagination to file uploads. auth.php handles login and registration logic. header.php and footer.php are the template pieces that get included on every page.

The /admin folder contains the entire administrative panel with its own includes subfolder for admin-specific templates. Each content type has its own management page.

The /user folder has the member dashboard pages for logged-in users.

And /assets holds all the static files organized into /css, /js, /images, and /uploads subfolders.

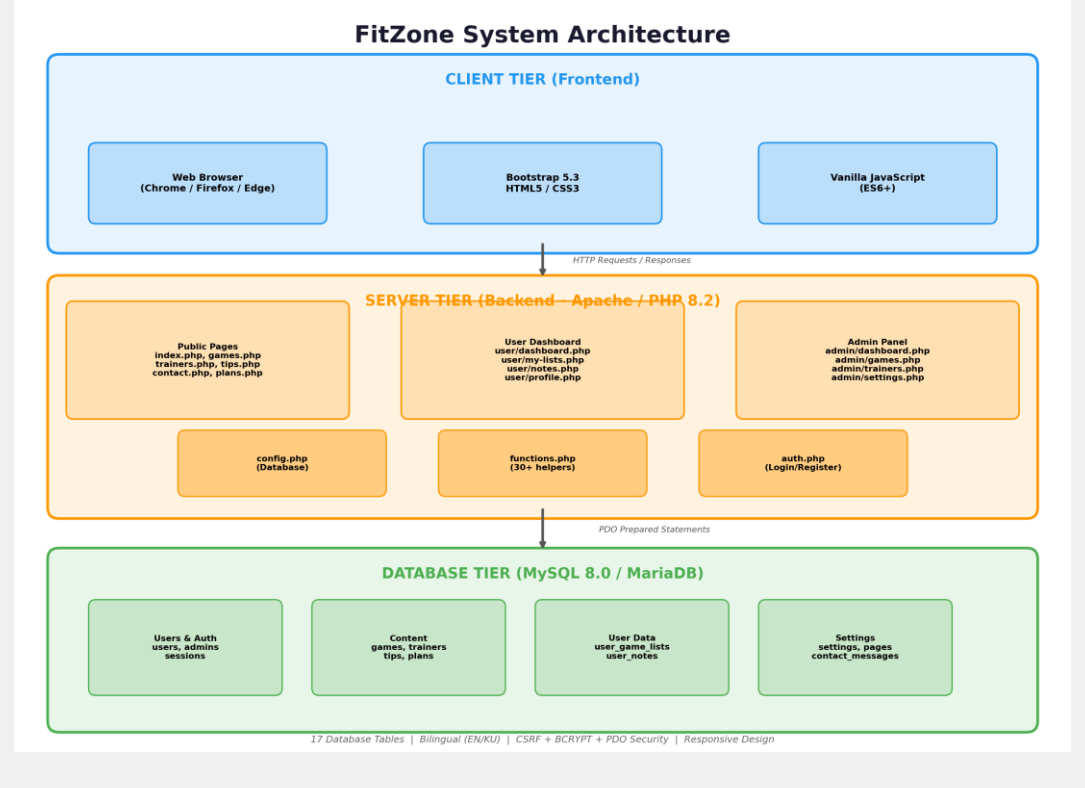

Figure 3.1: System Architecture Diagram

3.2.2 System Flowchart

The way users interact with FitZone depends on who they are. We designed three separate paths through the system:

Regular visitors (not logged in) can browse the homepage, look through exercises with filters, view trainer profiles, check membership plans, read fitness tips and articles, see gym certificates, and submit a contact form. They can also switch between English and Kurdish at any time.

Registered members get everything visitors get, plus access to their personal dashboard. From there they can create workout lists and add exercises from the database to those lists with custom sets and reps, write daily notes about their training with mood and weight tracking, and update their profile information.

Admins have their own separate login and dashboard. The admin dashboard shows key stats at a glance - total users, exercises, trainers, and unread messages. From there,

they can manage every piece of content on the site through dedicated management pages.

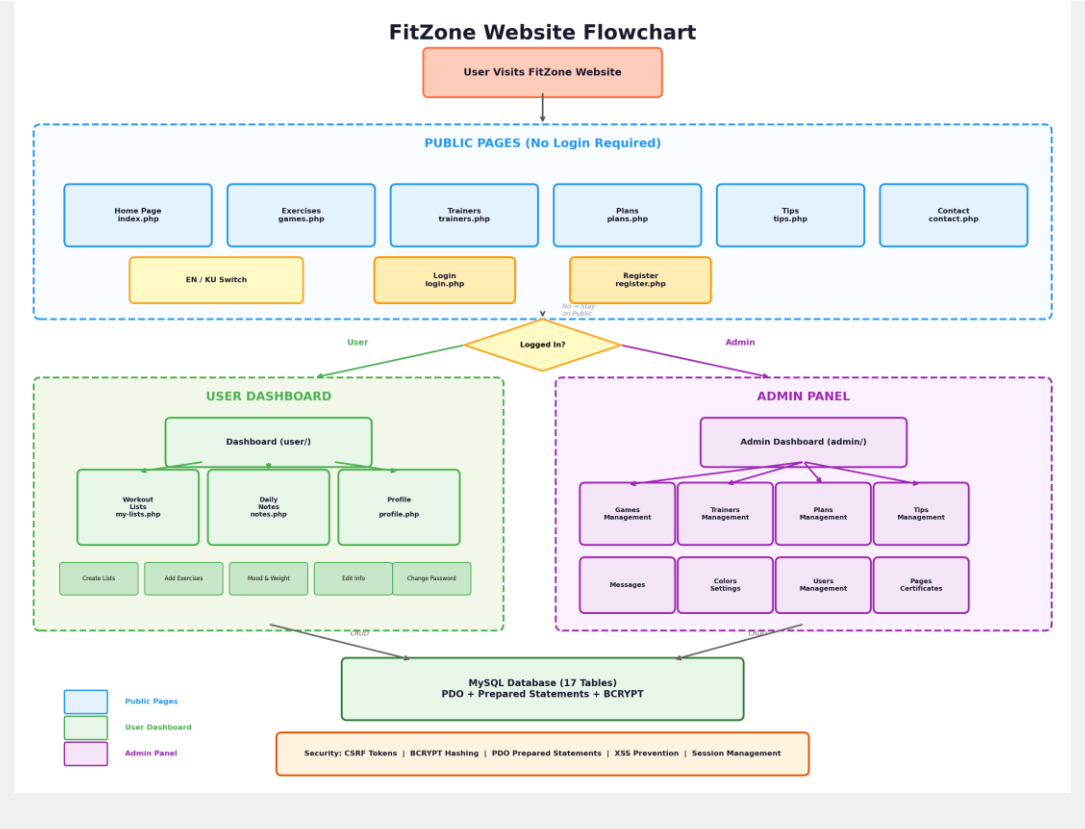

Figure 3.3: Website Flowchart

3.2.3 Database Design

The database was honestly one of the parts we spent the most time planning. We ended up with 17 tables, and getting the relationships between them right took a few tries. Here's what each table does:

Table Name	Description
users	All registered gym members - names in both languages, email, hashed password, avatar, bio, status
admins	Admin accounts with three roles: super_admin, admin, editor

games	Exercise database - names and descriptions in both languages, difficulty, muscles, equipment, video links
beginner_games	Pre-made beginner workouts organized by program, week, and day
beginner_programs	Training programs for beginners with duration and goal info
trainers	Trainer profiles with bios in both languages, certifications, contact info
tips	Articles and tips with categories like nutrition, exercise, lifestyle
certificates	Gym awards and certifications with type, year, and issuer
services	The services the gym offers, with descriptions and icons
plans	Membership packages - price, duration, features stored as JSON
reviews	Member testimonials with star ratings
contact_messages	Messages from contact form, with read/unread status
user_game_lists	Workout lists that users create for themselves
user_game_list_items	Individual exercises within a user's list
user_notes	Daily journal entries with mood, weight, and workout completion
pages	Custom pages that admins can create with their own URLs
settings	All site-wide settings, from site name to colors to social links

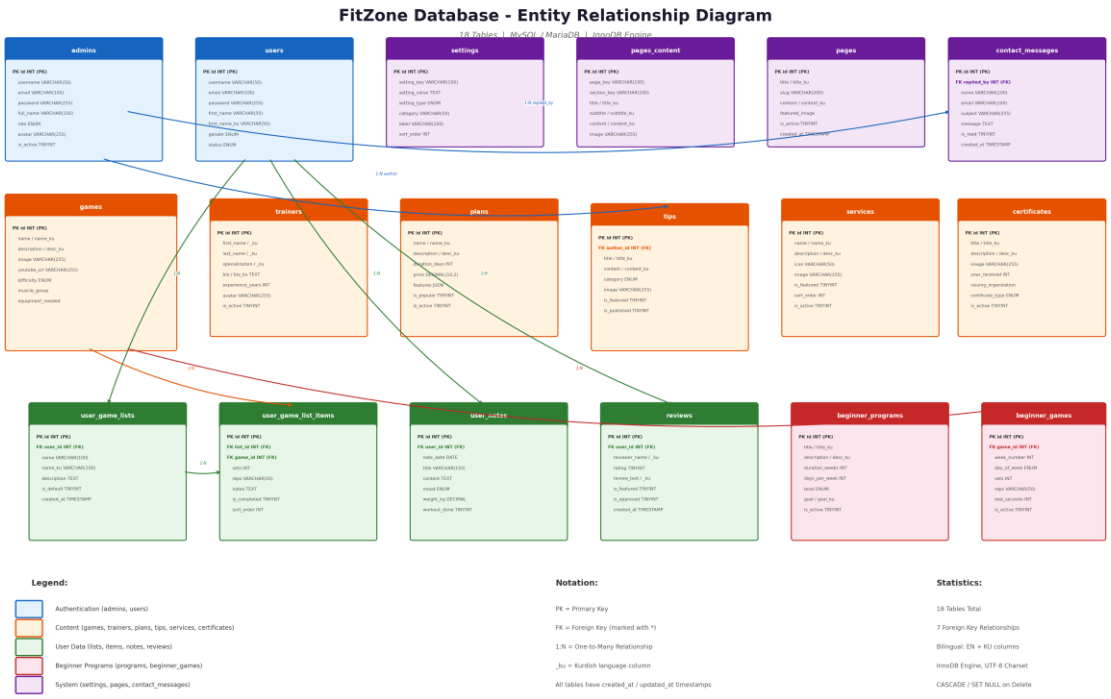

Figure 3.2: Database Entity-Relationship Diagram

Users Table Structure:

Since the users table is one of the most important ones, here's its complete structure:

Column	Type	Description
id	INT	Primary key, auto-increment
username	VARCHAR(50)	Unique username for login
email	VARCHAR(100)	Unique email address
password	VARCHAR(255)	BCRYPT hashed password
first_name	VARCHAR(50)	First name in English
last_name	VARCHAR(50)	Last name in English
first_name_ku	VARCHAR(50)	First name in Kurdish
last_name_ku	VARCHAR(50)	Last name in Kurdish

phone	VARCHAR(20)	Phone number
gender	ENUM	male, female, or other
date_of_birth	DATE	Birthday
avatar	VARCHAR(255)	Path to profile picture
bio / bio_ku	TEXT	About me text in both languages
status	ENUM	active, inactive, suspended, or banned
created_at	TIMESTAMP	When they signed up

3.3 Project Implementation

3.3.1 Database Configuration and Connection

One of the first things we built was the database connection setup in config.php. We used a design pattern called Singleton for this, which basically means there's only ever one connection to the database no matter how many times the code asks for it. This is more efficient than opening a new connection every time.

We went with PDO instead of the older mysqli extension. PDO is a bit more modern and it makes prepared statements really straightforward. We also set it up so that errors throw exceptions rather than failing silently, which makes debugging much easier.

On top of the database class, we wrote some simple wrapper functions - query() for running any SQL, fetchOne() for getting a single row, and fetchAll() for getting multiple rows. These made the rest of the code much cleaner since we didn't have to write out the full PDO prepare/execute pattern every time.

3.3.2 Authentication System

Getting the login and registration system right was really important to us. We've all heard stories about websites getting hacked because they stored passwords in plain text or didn't validate their inputs properly, and we didn't want to make those mistakes.

For registration, the registerUser() function in auth.php checks a bunch of things before creating an account: is the username already taken? Is the email already in use? Is the password at least 6 characters? Does it match the confirmation? If everything passes, it hashes the password using BCRYPT with a cost factor of 12, inserts the user into the database, and creates a default workout list for them.

Something we added that we're pretty happy about is that right after a new user registers, the system automatically creates a default workout list for them called "My Workout List." That way when they first log into their dashboard, they already have a list ready to add exercises to, which gives a better first experience.

For login, users can use either their email or username, which is a nice flexibility. The loginUser() function queries the database for a matching user, then uses password_verify() to check the password against the stored hash. If it matches, it creates session variables for the user's ID, username, and role.

Every single form on the site includes a CSRF token. This is a random string that gets generated and stored in the session, then embedded as a hidden field in the form. When the form is submitted, we compare the submitted token to the session token. If they don't match, we reject the submission. This prevents cross-site request forgery attacks.

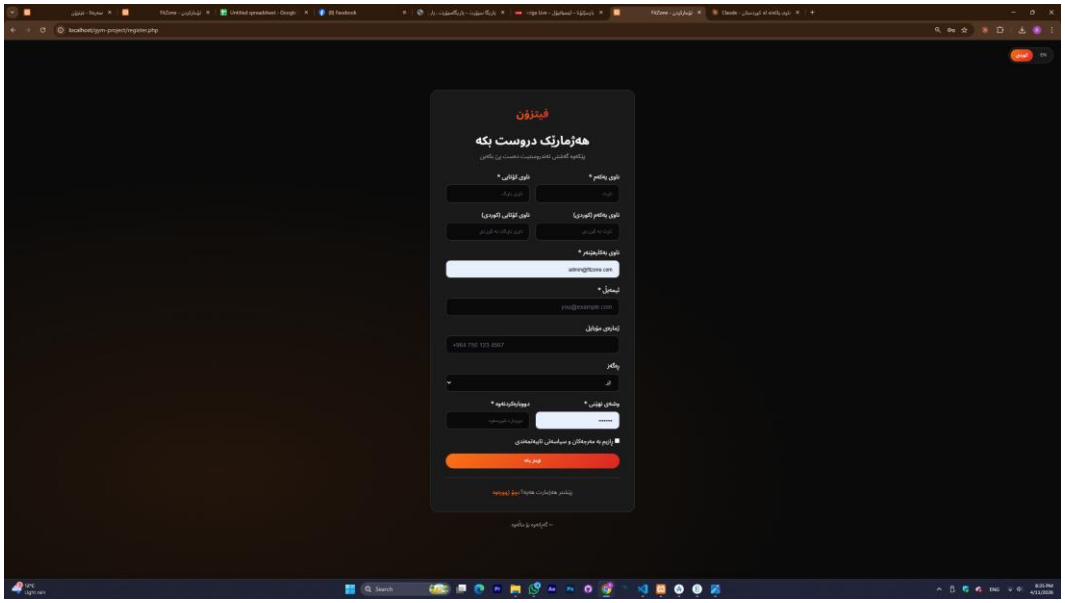

Figure 3.5: User Registration Page

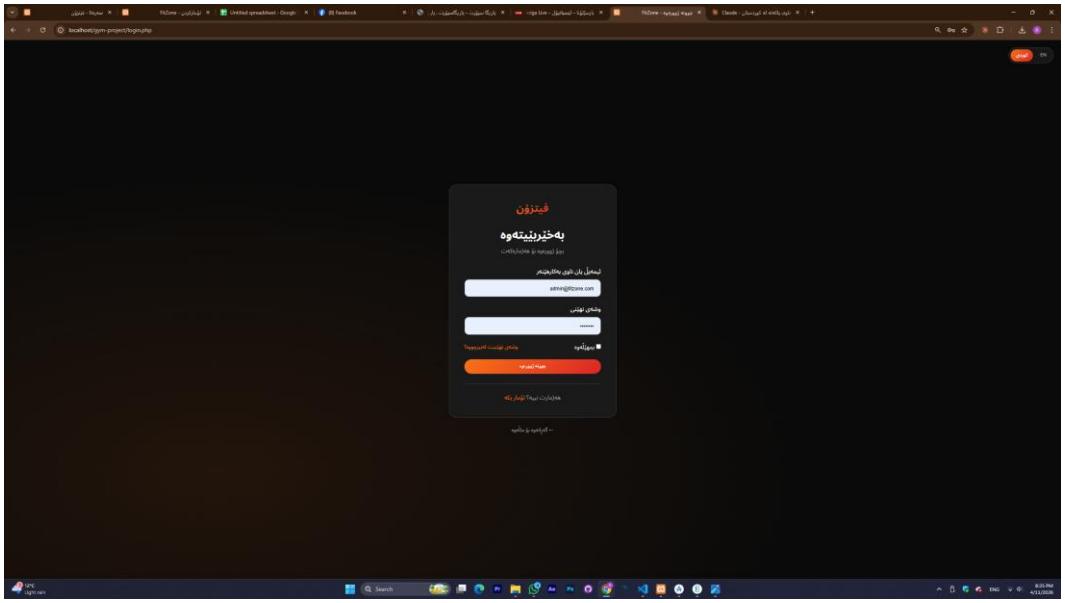

Figure 3.6: User Login Page

3.3.3 Public Website Pages

The homepage was the page we spent the most time tweaking visually. It has a big hero section at the top with a gradient background going from orange to red (our brand colors), a catchy headline, and a call-to-action button. Below that there are

sections for featured exercises, trainer spotlights, services, membership plans, and a call to action to join. All the text is bilingual.

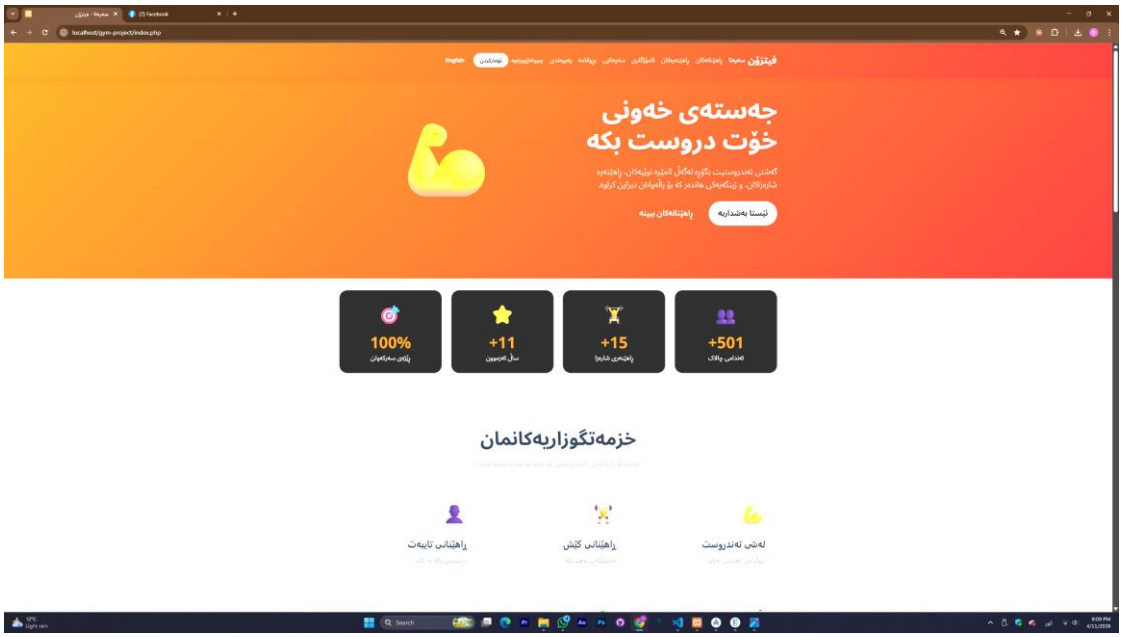

Figure 3.4a: Home Page - Hero Section, Stats & Services

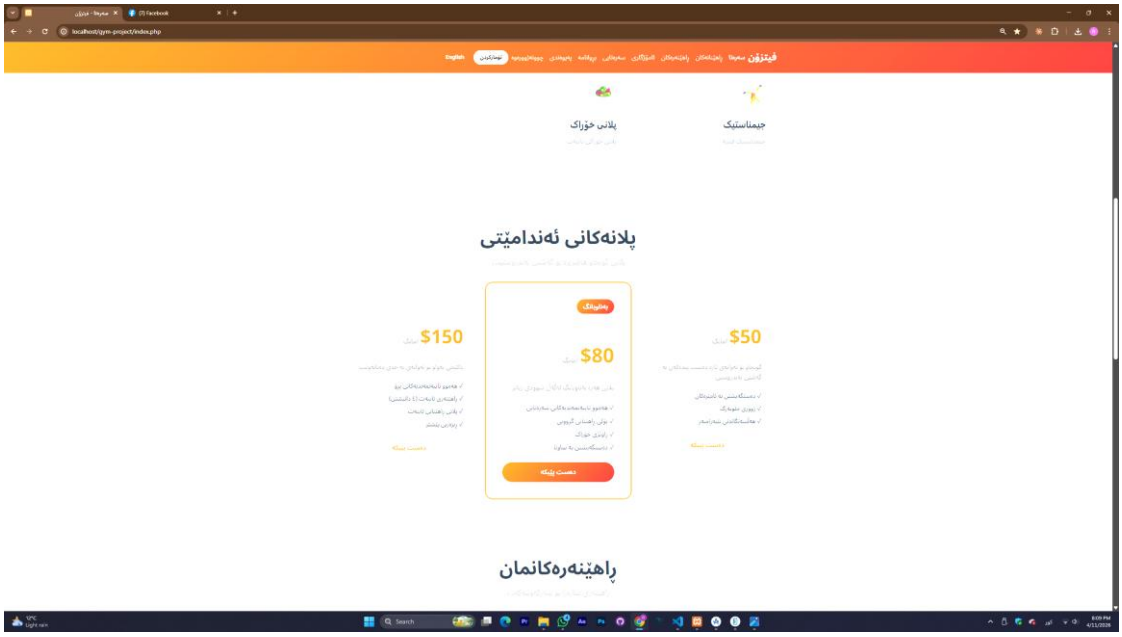

Figure 3.4b: Home Page - Membership Plans

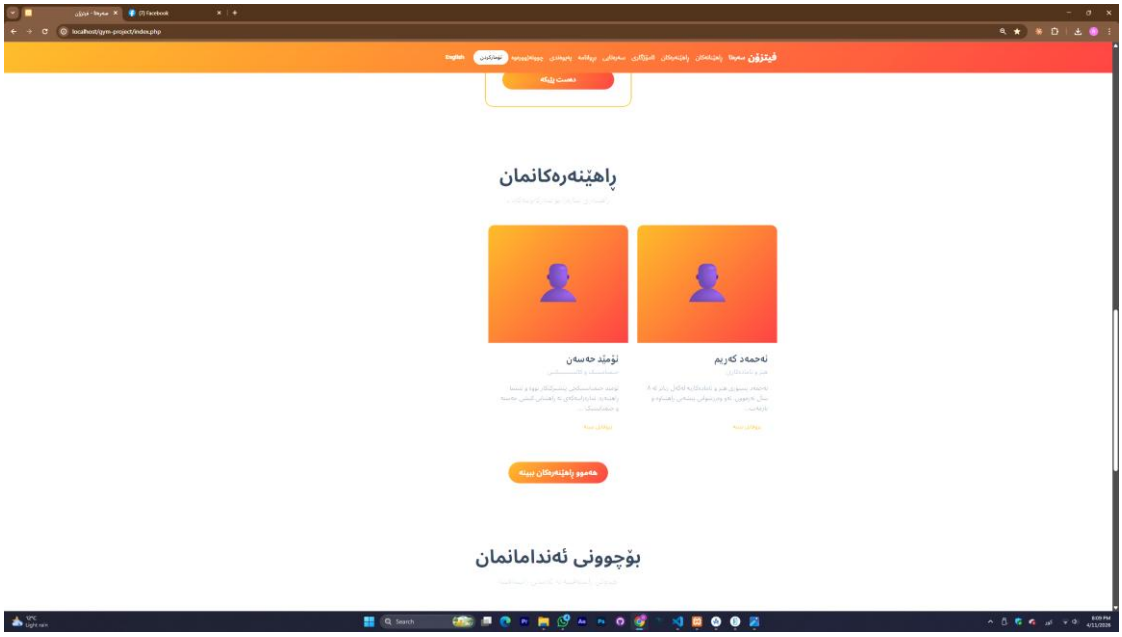

Figure 3.4c: Home Page - Trainers Section

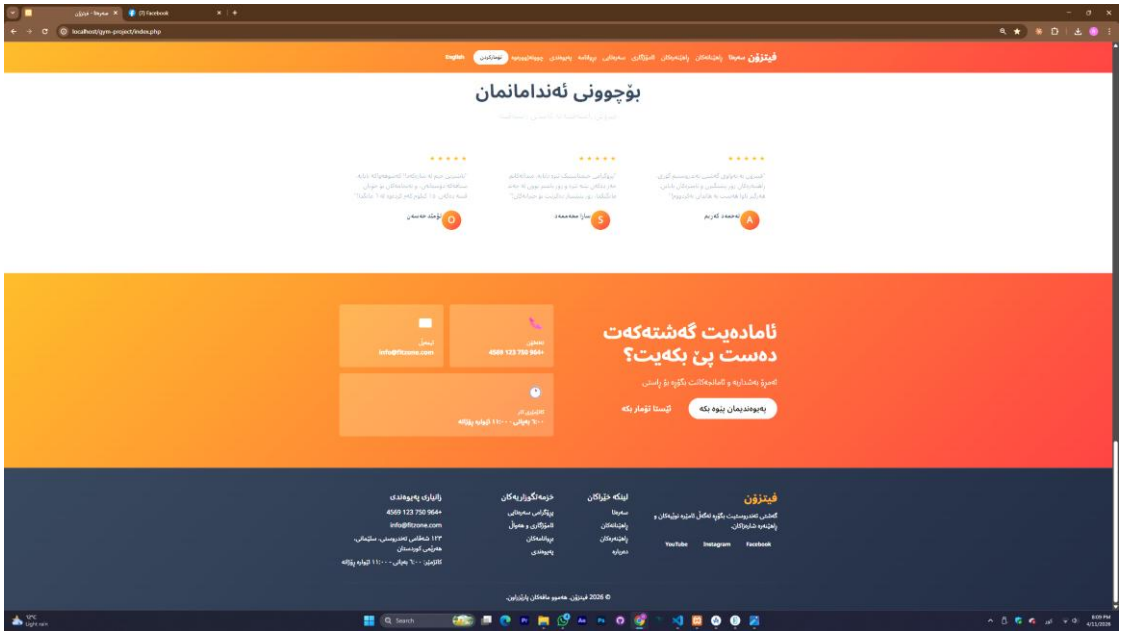

Figure 3.4d: Home Page - Reviews & Footer

The exercises page (games.php) is probably the most content-heavy page on the site. It pulls exercises from the database and displays them in a card layout. Each card shows the exercise name, a colored difficulty badge (beginner/intermediate/advanced), the target muscles, and a thumbnail. Users can filter exercises by difficulty, muscle group, or search by name. Clicking on a card takes you to a detail page with full descriptions in both languages, a YouTube video embed, equipment needed, duration, and tips.

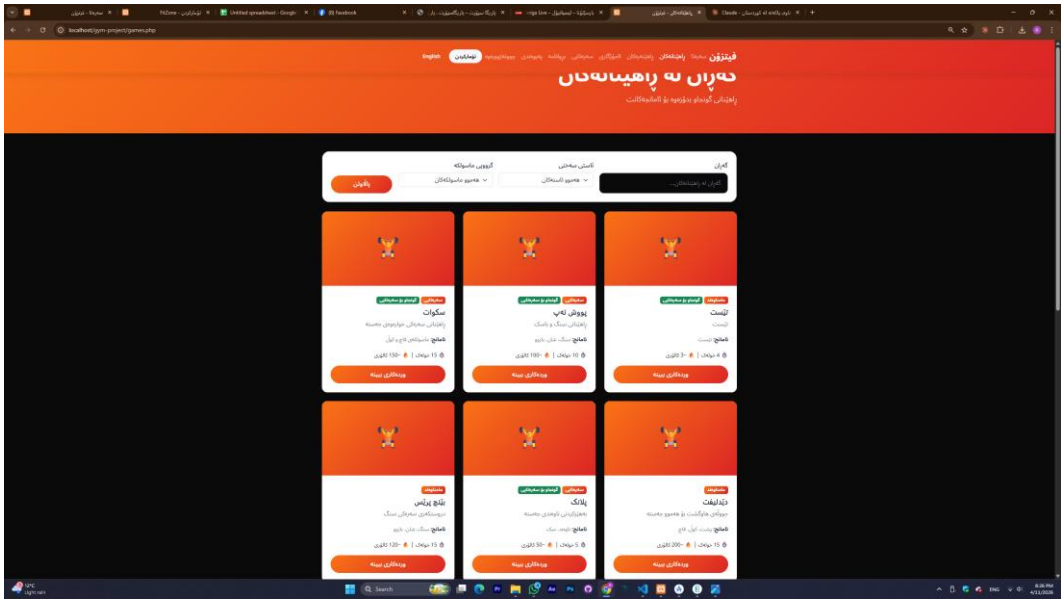

Figure 3.7: Exercises Page with Filters

The trainers page shows all active trainers in a nice grid. Each trainer has a card with their photo area, name, specialization, and a "View Profile" button. The detail page for each trainer shows their full bio in both languages, years of experience, certifications, and contact information.

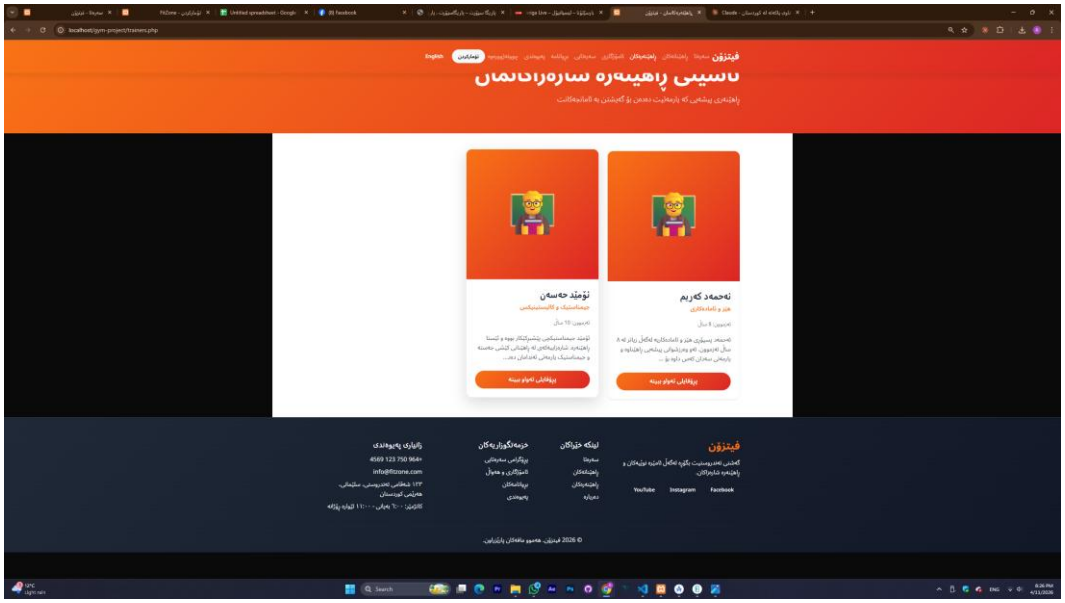

Figure 3.8: Trainers Page

The tips page works like a simple blog. Articles are categorized into nutrition, exercise, lifestyle, news, and motivation. You can filter by category, and each article has a featured image, title, excerpt, and full content in both languages. It supports featured articles that show up more prominently.

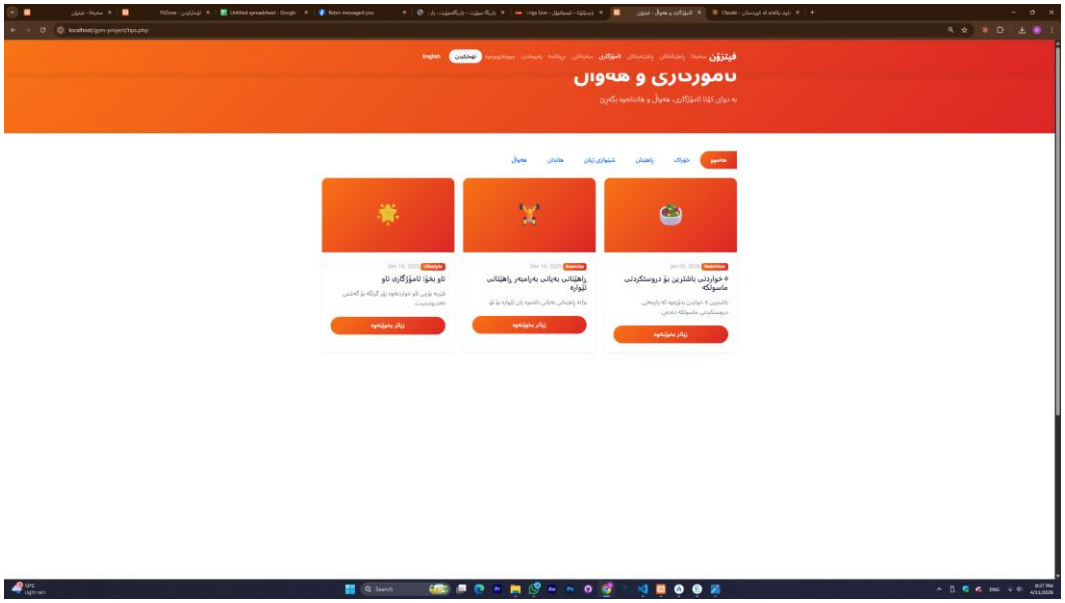

Figure 3.16: Tips / Blog Page

The contact page has a form where visitors can enter their name, email, phone, subject, and message. When submitted, the message gets stored in the contact_messages table and shows up in the admin's message inbox with an unread indicator.

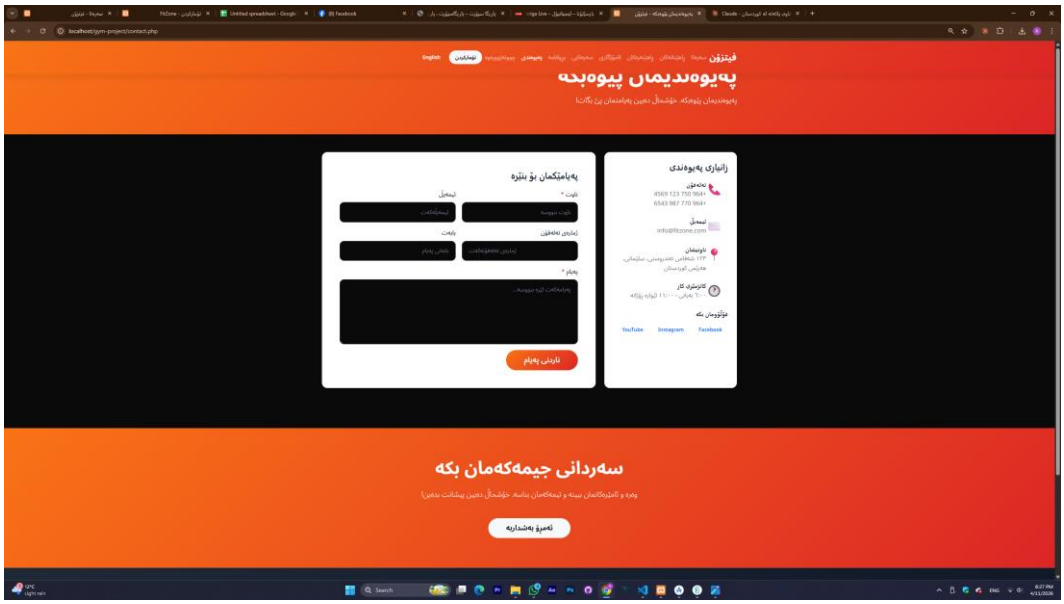

Figure 3.15: Contact Page

The membership plans page shows available plans in a card layout with pricing, duration, and a list of included features.

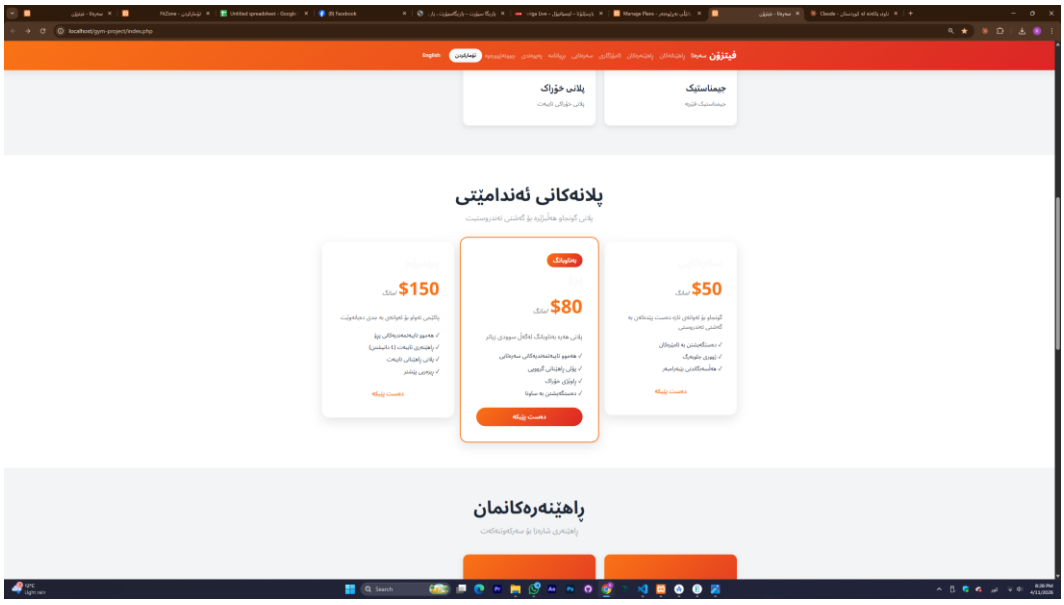

Figure 3.9: Membership Plans Page

3.3.4 User Dashboard

Once someone registers and logs in, they get access to their personal dashboard. This was actually one of the more fun parts to build because it's where the user can really interact with the site beyond just browsing.

The main dashboard page shows a welcome message with the user's name, some stat cards (how many workout lists they have, how many notes they've written, and when they joined), a quick-add form for writing today's note, and a list of their workout lists with exercise counts.

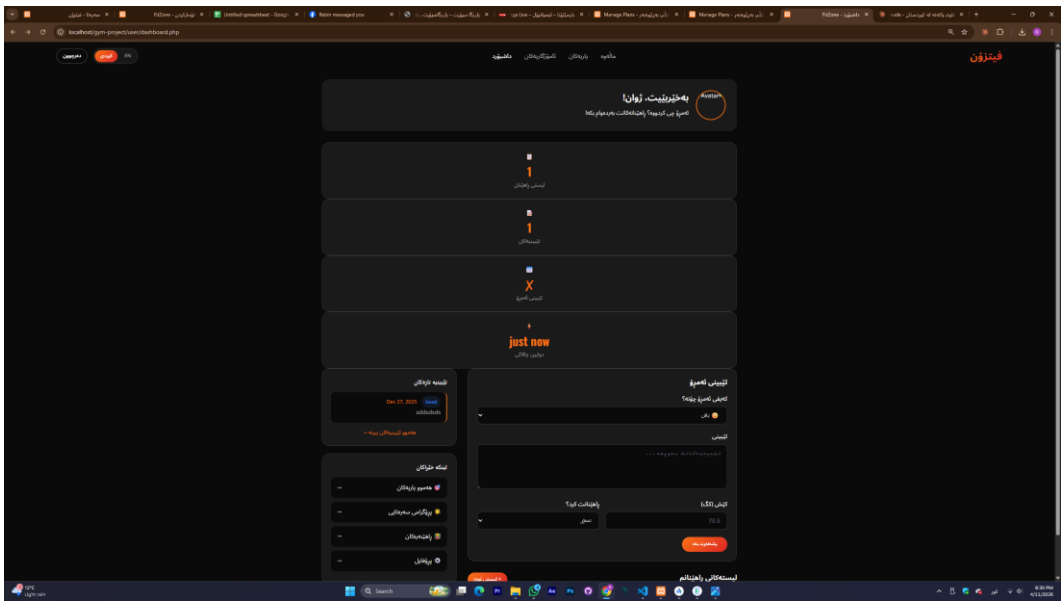

Figure 3.10: User Dashboard

The workout lists feature lets users create named playlists of exercises. For example, you could make a list called "Leg Day" and another one called "Upper Body." Then you can add exercises from the database to each list, specifying how many sets and reps you want to do. You can also rearrange the order or remove exercises.

The daily notes feature is like a workout journal. Each day you can write about your training, select your mood from five options (great, good, okay, tired, bad), log your weight in kilograms, and check off whether you worked out that day. There's a calendar view where you can browse past entries by month.

The profile page lets users update their name (in both English and Kurdish), email, phone number, date of birth, gender, and a short bio. There's also a password change section where they have to enter their current password before setting a new one.

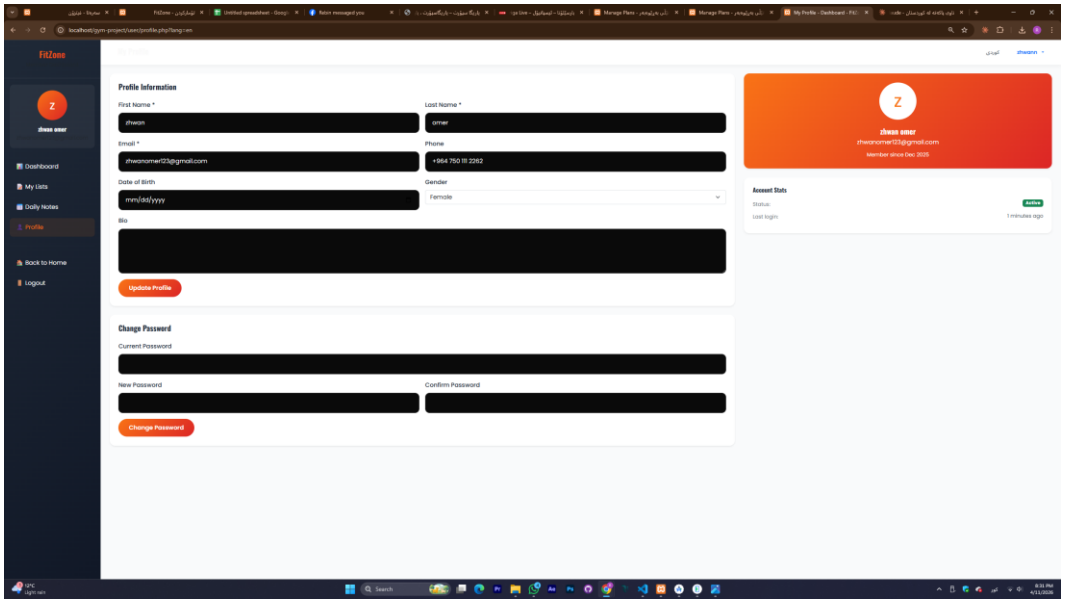

Figure 3.22: User Profile Page

3.3.5 Administrative Panel

The admin panel is arguably the biggest part of the whole project. It's where the gym owner or manager controls everything about the website. We gave it its own layout with a sidebar navigation on the left and the content area on the right, separate from the public website's design.

The admin dashboard is the first thing you see after logging in. It shows four main stat cards at the top: total users, active exercises, trainers, and unread messages. Each card has a growth indicator showing the trend. Below that there's a list of recent user registrations.

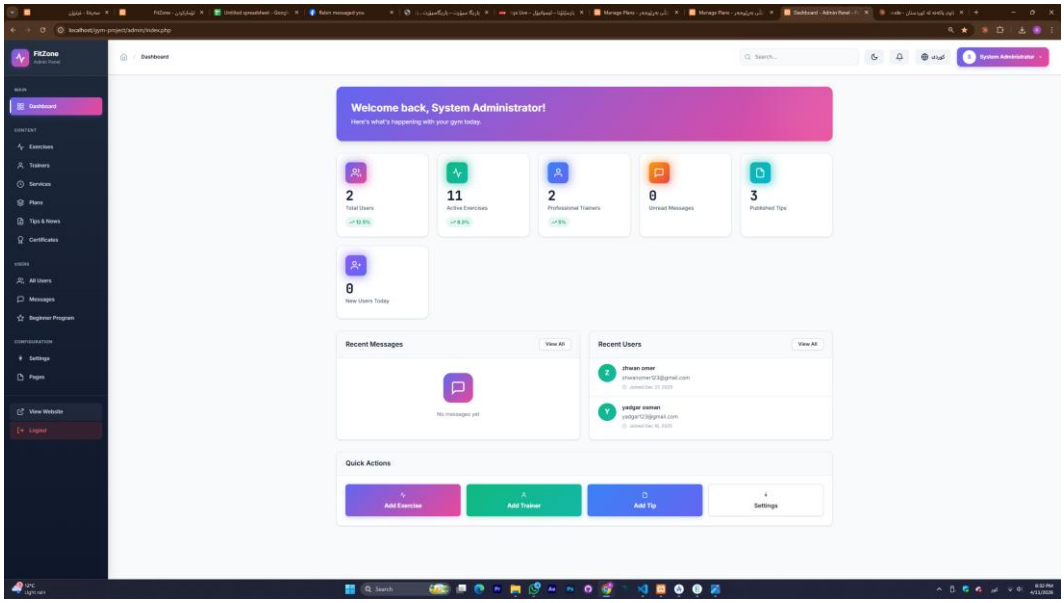

Figure 3.11: Admin Dashboard

For managing exercises, the admin can add a new exercise through a pretty big form. It asks for the name in English and Kurdish, a short description and full description in both languages, a YouTube video URL, an image upload, difficulty level selector, target muscles, equipment needed, estimated duration, calories burned, and sets/reps suggestions. The list view shows all exercises in a table with filters and a search bar.

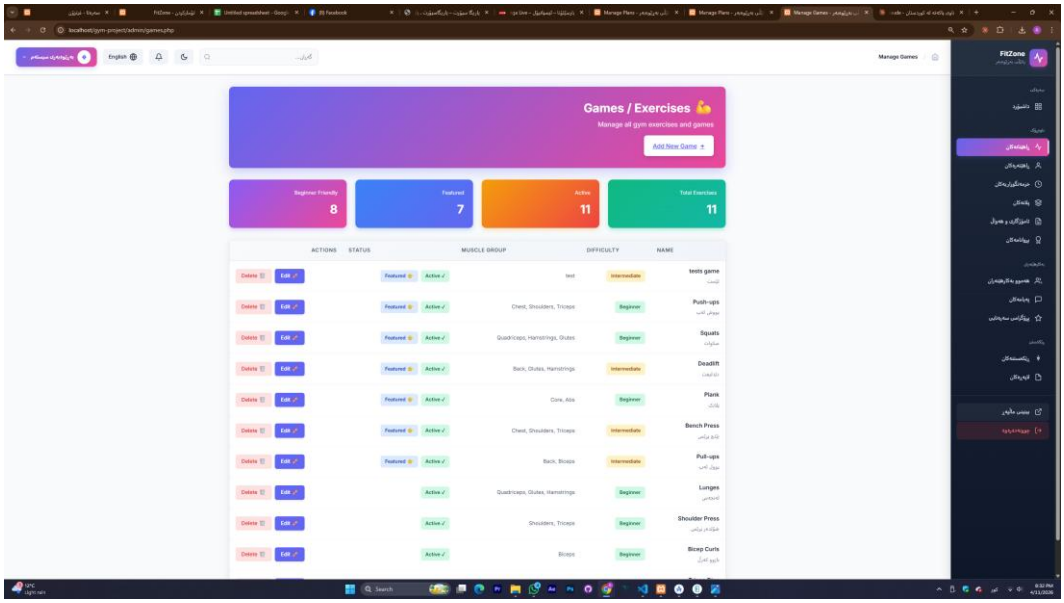

Figure 3.12: Admin Games Management

The trainers management page is similar - you can add trainers with all their info including a bio in both languages, their specialization, years of experience, certifications, phone and email, avatar upload, and a featured flag. The same CRUD operations (create, read, update, delete) are available.

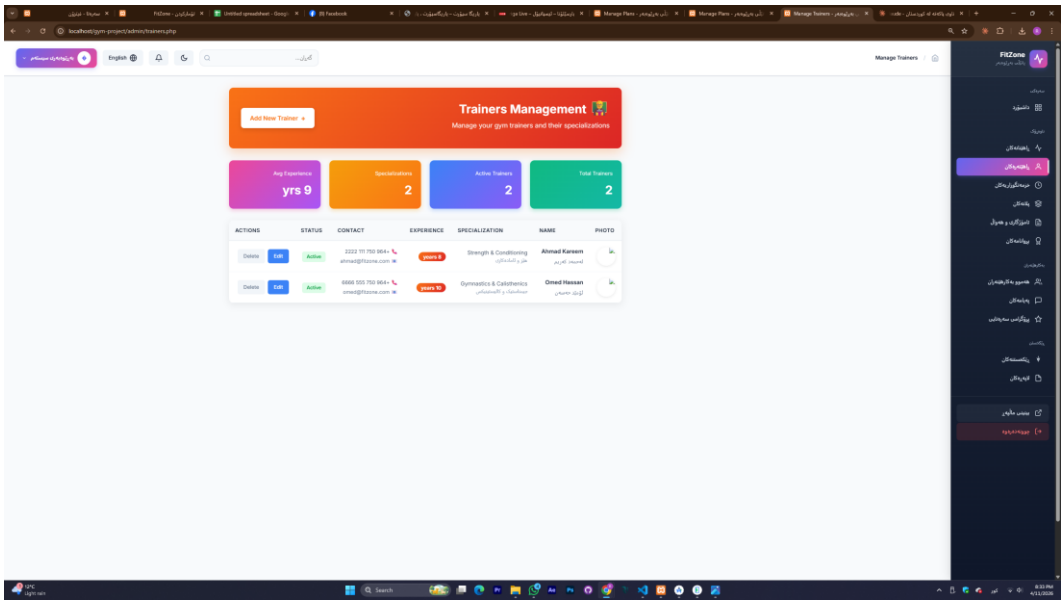

Figure 3.17: Admin Trainers Management

Membership plans are managed with fields for name, description in both languages, price, duration in days, and a text area for features where you enter one feature per line. The system stores these as JSON internally which makes them easy to display as a bullet list on the public site.

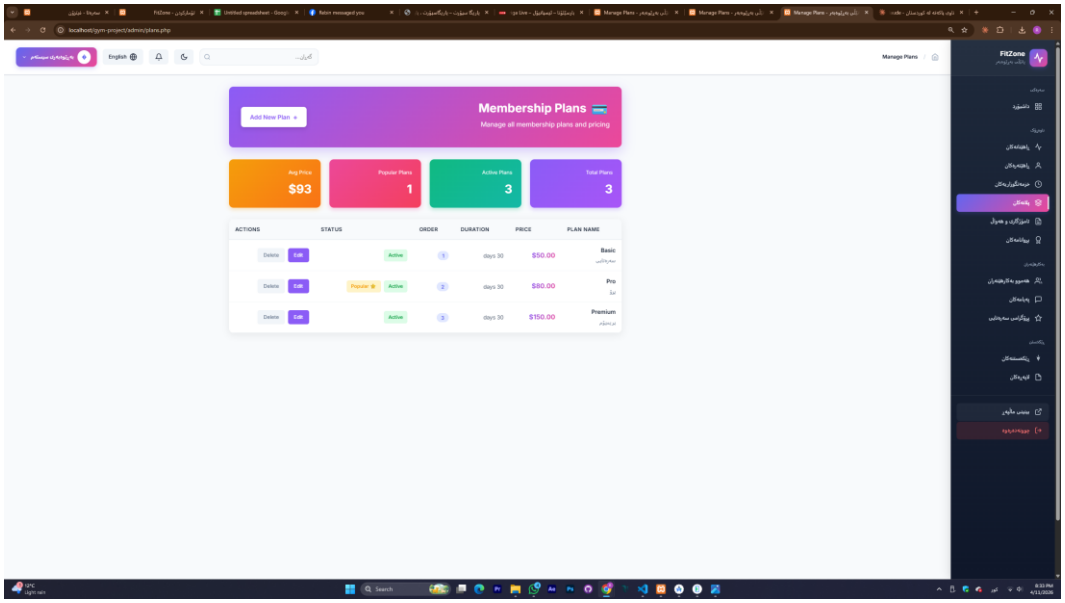

Figure 3.18: Admin Plans Management

The settings page is organized into tabs: General (site name, description, hero text), Contact (email, phone, address), Social (social media links), and Colors. The colors section has over 45 options for customizing every color on the site - from primary and secondary brand colors to text colors, background colors, button colors, and more.

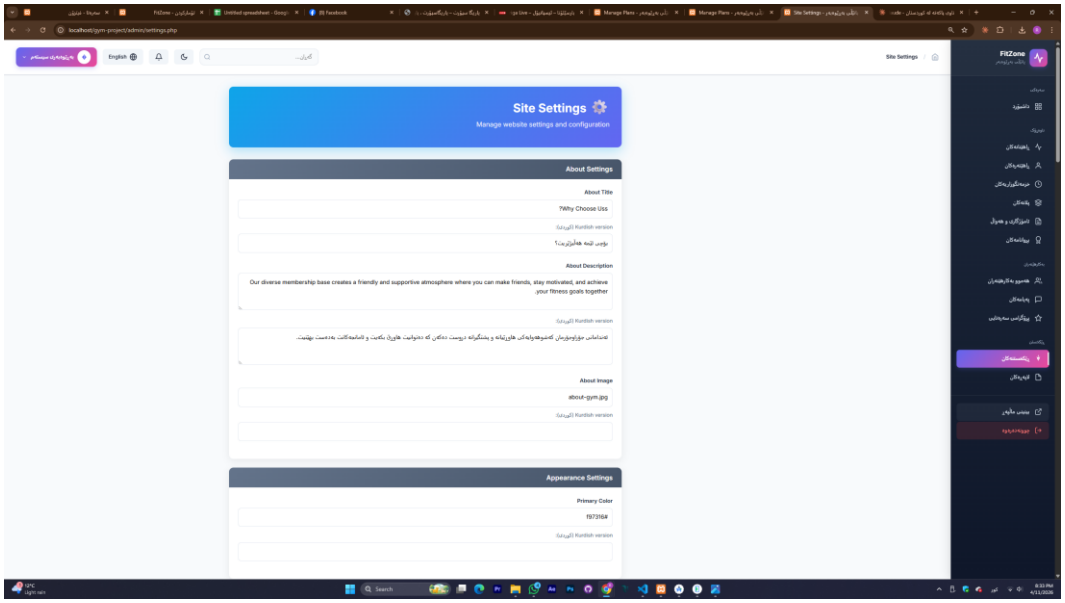

Figure 3.13: Admin Settings Page

The messages page shows all contact form submissions in a table with the sender's name, email, subject, and whether it's been read. Clicking on a message shows the full details and automatically marks it as read.

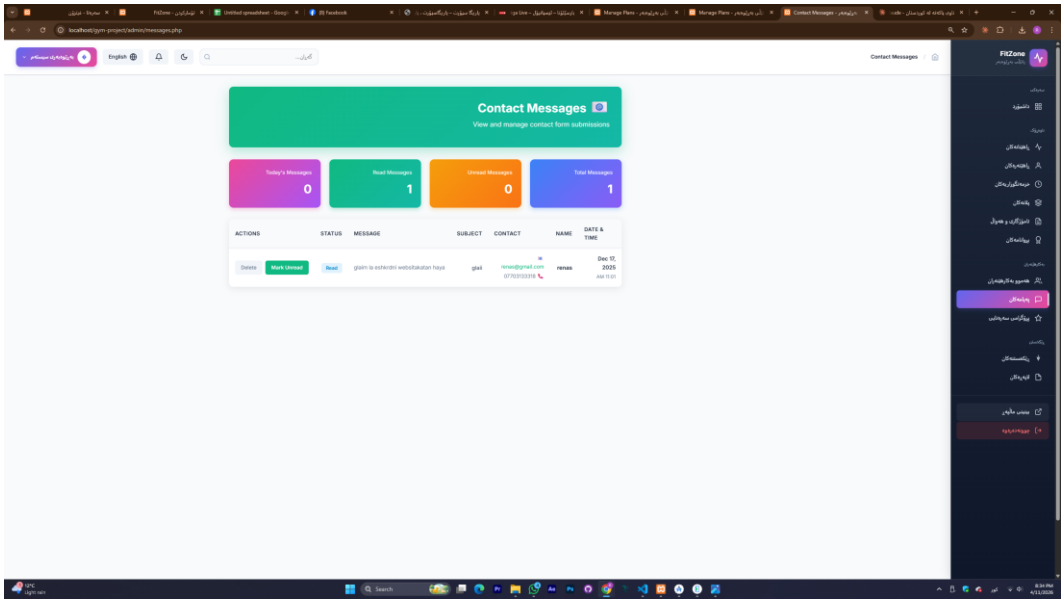

Figure 3.19: Admin Messages Page

3.3.6 Bilingual Support System

Making the site work in both Kurdish and English was probably the most challenging part of the whole project, mainly because of the RTL layout. When you switch to Kurdish, everything needs to flip - text alignment, navigation direction, form layouts, even the sidebar in the admin panel.

We handled language switching through a simple system: there's a toggle in the navbar that lets users switch between EN and KU. When they click it, the language preference gets saved in the session (so it persists across pages) and optionally in a cookie (so it persists across visits). The page then reloads with the new language.

For text that's hardcoded in the PHP files (like button labels and headings), we created a helper function called __() (double underscore). You call it like __("Join Now", "بەشدار بە ئێستا") and it returns the appropriate string based on the current language setting.

For content stored in the database, like exercise names or trainer bios, we use duplicate columns. So the games table has both a "name" column and a "name_ku" column. The getLocalized() function figures out which column to use based on the current language.

For fonts, we had to use Google's Noto Sans Arabic font alongside the regular Inter and Oswald fonts, because Kurdish script needs proper Arabic-script font support to look right.

3.3.7 Security Implementation

Security was something we tried to think about from the start rather than adding it as an afterthought. Here's what we did:

- CSRF tokens on every form - we generate a random token, store it in the session, embed it as a hidden field, and verify it when the form comes back. If it doesn't match, the form gets rejected.
- Passwords are never stored in plain text. We use PHP's password_hash() with BCRYPT and a cost factor of 12. Even if someone got access to the database, they'd just see hashed strings that would take years to crack.
- All database queries use prepared statements through PDO. This means user input is never directly inserted into SQL strings, which completely prevents SQL injection. We never concatenate user input into queries.
- Output escaping everywhere - any time we display user-supplied data on a page, it goes through our e() function which calls htmlspecialchars(). This prevents XSS attacks where someone might try to inject JavaScript through a form field.
- File uploads are restricted to image types only (jpg, jpeg, png, gif, webp) and limited to 5MB. Uploaded files get renamed to random strings so attackers can't predict file paths or overwrite existing files.

- User and admin sessions are completely separate. Even if somehow a regular user session got elevated, they couldn't access admin functions because the code checks for admin-specific session variables.
- We validate inputs on the server side even though there's also client-side validation in JavaScript. Client-side validation can always be bypassed, so the server checks are what really matter.

3.3.8 Responsive Design

Making the site work well on different screen sizes was something we tested throughout development, not just at the end. We'd regularly pull up Chrome's device toolbar and check how things looked on phone, tablet, and desktop widths.

Bootstrap's grid system did most of the heavy lifting here. We used classes like col-lg-4 col-md-6 to make cards go from three columns on desktop to two on tablet to one on mobile. The navigation collapses into a hamburger menu on smaller screens. Tables scroll horizontally when they're too wide.

We also used CSS custom properties (variables) for a lot of the styling. This made it easy to keep colors and spacing consistent, and it's also what powers the admin color customization feature since changing a variable value in CSS automatically cascades to every element that uses it.

3.4 Project Publishing

Right now FitZone runs locally through XAMPP, which works perfectly for development and demonstration. But it's designed to be deployed to a real web server too. Any hosting that supports PHP 8.0 or higher and MySQL/MariaDB will work - which is basically any shared hosting plan out there.

To deploy it, you'd basically need to upload all the project files to the hosting via FTP or a file manager, create a new database and import our SQL setup files through

phpMyAdmin (which most hosts provide), update the database credentials in config.php, and you're good to go. We wrote an installation guide that walks through the whole process step by step.

We think FitZone would work well for any small to medium sized gym that wants an online presence. The bilingual feature makes it especially useful for gyms in the Kurdistan Region since their members speak both Kurdish and English. The admin panel is user-friendly enough that a non-technical gym owner could manage it after a quick walkthrough.

CHAPTER FOUR

CONCLUSION AND FUTURE WORK

4.1 Conclusion

Looking back at what we've built, we're honestly pretty proud of how FitZone turned out. When we started this project, we had a rough idea of what we wanted but we weren't sure we could pull it all off in the time we had. Turns out we could - mostly.

The main thing we set out to do was create a bilingual gym website that works in both Kurdish and English, and we accomplished that. Every page, every piece of content, every button and label can be viewed in both languages with proper RTL layout for Kurdish.

Here's a quick summary of what we achieved:

- A fully working bilingual website with both English and Kurdish, including proper RTL layout that we tested extensively to make sure nothing looked broken when switching languages.
- An exercise database system that can hold hundreds of exercises with all the details a gym-goer would want - difficulty, muscles worked, equipment, video demonstrations, instructions, and tips.
- A secure authentication system that we're fairly confident in. Passwords are properly hashed, forms are protected against CSRF, queries are parameterized, and all output is escaped.
- A user dashboard where members can create their own workout playlists, keep a daily training journal, and manage their personal information.

- A comprehensive admin panel with management interfaces for 10 different types of content, plus a settings system with over 45 customizable color options.
- Responsive design that we tested on various screen sizes and it looks good on everything from a large monitor down to a phone screen.
- A clean codebase with reusable helper functions, a singleton database connection, and organized file structure that would make it reasonably easy for another developer to pick up and work on.

Were there challenges? Absolutely. The biggest one was the bilingual system - it's easy to make a page display in two languages, but making sure forms, error messages, flash notifications, and dynamic content all switch correctly took a lot of careful testing.

All in all, this project taught us way more than just PHP and MySQL. We learned how to plan a real software project, work as a team, make design decisions with trade-offs, and push through when things got frustrating. Those are skills that'll serve us well regardless of what technology we work with next.

4.2 Future Works

There are quite a few things we would love to add to FitZone if we had more time. Some of these are features we planned from the start but couldn't fit into the timeline, and others are ideas that came up during development:

- Online payments - right now the plans page just shows prices, but it would be great if members could actually pay through the website using services like FIB or FastPay that people in Kurdistan use.
- A mobile app would be the natural next step. We could build one with React Native or Flutter that connects to the same database and lets users track their workouts from their phone without opening a browser.

- Weight progress charts - we already collect weight data in the daily notes, so it would be cool to show users a graph of how their weight has changed over time. Same for workout frequency.
- A notification system that sends emails for things like membership renewals coming up, inactivity reminders, or new tips posted.
- Video uploads for trainers - right now exercises link to YouTube, but it would be better if trainers could upload their own demonstration videos directly.
- A real-time chat feature so members could message trainers directly through the site for quick advice or scheduling.
- QR code check-in for gym attendance tracking. Members would scan a code when they arrive and the system would record their visit.
- Some kind of AI recommendation system that suggests exercises based on the user's goals, fitness level, and past workout history.
- Support for multiple gym branches, so a gym chain could manage all their locations from one system.
- Adding Arabic as a third language, which would let the system be used in other parts of Iraq too.

REFERENCES

[1] Nixon, R. (2021). Learning PHP, MySQL & JavaScript: A Step-by-Step Guide to Creating Dynamic Websites. 6th Edition. O'Reilly Media.

[2] Duckett, J. (2014). HTML and CSS: Design and Build Websites. John Wiley & Sons.

[3] Duckett, J. (2014). JavaScript and jQuery: Interactive Front-End Web Development. John Wiley & Sons.

[4] Welling, L. and Thomson, L. (2016). PHP and MySQL Web Development. 5th Edition. Addison-Wesley Professional.

[5] Spurlock, J. (2013). Bootstrap: Responsive Web Development. O'Reilly Media.

[6] W3Techs (2024). Usage Statistics of Server-side Programming Languages for Websites. Available at: https://w3techs.com/technologies/overview/programming_language

[7] PHP Documentation (2024). PHP: Hypertext Preprocessor. Available at: https://www.php.net/docs.php

[8] MySQL Documentation (2024). MySQL 8.0 Reference Manual. Available at: https://dev.mysql.com/doc/refman/8.0/en/

[9] Bootstrap Documentation (2024). Bootstrap 5 Documentation. Available at: https://getbootstrap.com/docs/5.3/

[10] MDN Web Docs (2024). JavaScript Reference. Mozilla Developer Network. Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript

[11] OWASP Foundation (2023). OWASP Top Ten Web Application Security Risks. Available at: https://owasp.org/www-project-top-ten/

[12] Apache Friends (2024). XAMPP - Apache + MariaDB + PHP + Perl. Available at: https://www.apachefriends.org/